

Campus Log Anomaly Detection under Proxy Labels: Validity Limits of Archive Splits and Masked TCN–Transformers

Tien Thanh Cao^{1*}, Hoang Tu Trinh¹, and Manh Ha Tran¹

Faculty of Information Technology,
Ho Chi Minh City University of Foreign Languages – Information Technology,
Ho Chi Minh City, Vietnam
thanhct@huflit.edu.vn, 23dh113972@st.huflit.edu.vn, hatm@huflit.edu.vn

Abstract. Archive ordering and proxy window labels can distort campus log anomaly evaluation. On selective HUFLIT extracts, we compare (A) an archive-order stream-position protocol with validation contamination and (B) a 1,000,000-line exact-deduplicated, source-bounded, backup-day-held-out protocol, using a custom similarity parser and HTTP/keyword *proxy* labels. Under Protocol A, test-window prevalence reaches 80.5% because a window is positive if any of $L=20$ lines is positive; an always-anomaly classifier then obtains $F1 \approx 0.892$, while a masked TCN–Transformer under $\tau = \mu + 3\sigma$ scores only 0.0395 ± 0.0163 F1 despite ROC-AUC 0.7226 ± 0.0178 (AUPRC lift over prevalence $\approx +0.031$). Under Protocol B (train 15–17 Jun, val 18 Jun, test 19 Jun 2026 backup days), test prevalence falls to 56.4% (always-anomaly F1 baseline 0.721); a DeepLog-style LSTM retains the strongest mean AUPRC lift (+0.138) but with large seed variance, masked models collapse near chance, and PCA stays more stable (+0.098). Keyword/status template ablation only partially reduces PCA lift ($+0.098 \rightarrow +0.084$). Threshold comparison shows that $\mu + 3\sigma$ suppresses recall, whereas validation-selected thresholds raise F1 toward the high-prevalence trivial baseline. Under Protocol B, the TCN increases batch-1 PLL latency from 18.5 to 51.1 ms without a consistent accuracy benefit.

Keywords: AIOps · Log Anomaly Detection · Proxy Labels · Experimental Validity · TCN–Transformer · Campus Infrastructure.

1 Introduction

Campus IT services emit heterogeneous operational logs. Sequence detectors can score unusual template patterns [2–5], but archive packaging, duplicate backups, and heuristic labels can invent “temporal” structure that models then appear to learn. Prior empirical studies already show that training-data selection, grouping, class distribution, and noise strongly affect public LAD benchmarks [9, 10].

* Corresponding author: thanhct@huflit.edu.vn

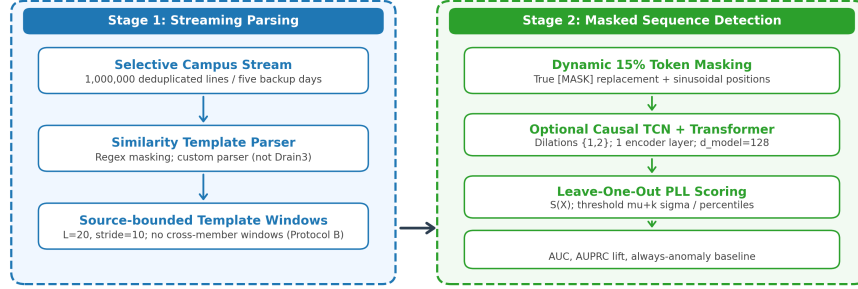


Fig. 1. Measured detector pipeline: streaming parsing and masked PLL scoring.

We extend that validity agenda to a setting those studies do not isolate: institutional *backup archives* whose folder order is not event time, with exact raw-line contamination audits, OR-window prevalence inflation, proxy-label/feature coupling, and negative evidence for a masked TCN–Transformer.

This paper treats those threats as first-class experimental objects. We study a masked TCN–Transformer and a no-TCN ablated Transformer on HUFLIT selective extracts under two protocols: (A) an archive-order stream-position split with validation contamination; (B) a jointly decontaminated protocol (exact-line deduplication, boundary-safe windows, and backup-day holdout). We add trivial baselines (always-anomaly; no-skill AUPRC = prevalence), report AUPRC lift over prevalence within each protocol, and compare fixed versus validation-selected thresholds. We do *not* claim an LLM root-cause system.

The contributions are:

- **Validity protocol on campus archives:** measure OR-window prevalence inflation against an always-anomaly trivial baseline, audit exact raw-line and template-window contamination, and retain dual-protocol artifacts (`huflit_baselines.json`, `huflit.v2.*`).
- **Joint decontamination protocol:** Protocol B applies exact-line SHA-1 dedup, windows that never cross (backup-day, source, member), and backup-day holdout; DeepLog-style and LogAnomaly-lite baselines are evaluated on the same arrays. We do *not* claim a causal decomposition of the three factors.
- **Threshold-aware reading:** fixed $\mu+3\sigma$ versus validation-selected F1 thresholds with precision/recall, separating ranking from calibration.
- **Negative TCN evidence:** on both protocols the TCN does not consistently beat a masked Transformer without TCN, while increasing leave-one-out scoring latency; keyword/status ablation shows only partial proxy-token leakage into PCA lift.

2 Related Work

2.1 Log Parsing and Template Extraction

Log parsing transforms free-form messages into event templates by separating constants from dynamic parameters [1, 12]. Drain uses a fixed-depth parse

tree [1]; Spell uses streaming sequence matching [12]. Our artifact does *not* implement Drain or Drain3. It applies regex masking and a linear scan over equal-length templates, merging tokens when positional similarity exceeds 0.5. We call this the *custom similarity parser* and avoid attributing Drain3 properties or throughput to it.

2.2 Deep Learning for Log Anomaly Detection

Sequence-based anomaly detection models learn temporal transition dynamics over sequences of log templates [2, 3, 6]. Du et al. [2] introduced DeepLog (LSTM next-token prediction). Meng et al. [3] proposed LogAnomaly with semantic template vectors. SwissLog [5] and LogRobust [4] address instability and interleaved threads. Classical reconstruction baselines such as PCA on count vectors remain competitive on some structured logs [6]. Recent Transformer–TCN hybrids [15], multi-scale semantic models [13], and contrastive masked objectives [14] continue this line. Public corpora such as Loghub [11] remain the standard sanity setting; we defer a full HDFS/BGL re-run under the present harness to future work.

2.3 Experimental Validity in Log Anomaly Detection

Le and Zhang [9] show that training-data selection, grouping, class distribution, data noise, and early-detection settings can overturn optimistic LAD conclusions on public benchmarks. Albert [10] argues that training should precede testing chronologically and analyzes grouping, anomaly ratio, and duplicate sequences. Our contribution is complementary rather than redundant: we study an institutional campus *backup archive* where packaging order is not wall-clock event time; we report exact raw-line hash contamination and OR-window prevalence inflation with an always-anomaly baseline; we probe proxy-label/feature coupling via template ablation; and we provide negative evidence for a masked TCN–Transformer under that archive protocol.

Table 1. Qualitative comparison of related detectors. Latency is *not* cross-comparable: this work reports batch-1 leave-one-out PLL; DeepLog/LogAnomaly-lite legacy batched scoring is NR for batch-1.

Approach	Parser	Detector	Focus	CPU ms
DeepLog [2]	Spell	LSTM next-token	Sequence LAD	NR [†]
LogAnomaly [3]	FT-Tree + template2Vec	LSTM (seq. + quant.)	Sequence LAD	NR ^{†,‡}
LogRobust [4]	Drain	Bi-LSTM+Attn	Unstable logs	NR
SwissLog [5]	Online	Attentive Bi-LSTM	Interleaved logs	NR
Le & Zhang [9]	Drain	Multiple DL	Public-benchmark validity	NR
Albert [10]	Drain	DeepLog-based	Chronological LAD protocol	NR
This work	Custom similarity	Masked TCN–Transformer	Campus-archive validity	51.1 [§]

[†]Legacy batched ms retained in JSON but not relabeled as batch-1 PLL. [‡]LogAnomaly-lite is our reimplement on indexed templates and does not reproduce the original template2Vec semantic representation or the quantitative (count) branch; scoring uses mean token reconstruction NLL only. [§]Batch-1 leave-one-out PLL on Protocol B (CPU, 2 threads).

3 Methodology

3.1 System Architecture Overview

The measured system has two stages: (i) **Parsing**: selected syslog messages are tokenized by the custom similarity parser; (ii) **Masked sequence scoring**: sliding template windows are corrupted by true token replacement, encoded by an optional causal TCN plus Transformer, and scored by leave-one-position-out pseudo-log-likelihood (PLL).

3.2 Masked TCN-Transformer Denoising Sequence Model

We formulate the detector as a masked template language model: 15% of input tokens are replaced by a reserved mask token, representations pass through a Temporal Convolutional Network (TCN) [7] and Transformer self-attention [8], and a linear vocabulary head predicts the original identities only at selected positions.

Embedding and Dynamic Masking A sliding window of length $L = 20$ with stride $S = 10$ extracts template sequences $\mathbf{X} = [T_1, T_2, \dots, T_L]$. During training, 15% of tokens are replaced with [MASK] ($ID = |\mathcal{V}| + 1$), distinct from padding ($ID = 0$). Padding masks are applied in self-attention. Embeddings use sinusoidal positional encodings [8]:

$$\mathbf{E} = \text{Embedding}(\mathbf{X}) + \mathbf{P}_{\text{pos}} \in \mathbb{R}^{L \times d_{\text{model}}} \quad (1)$$

Temporal Convolutional Network (TCN) Layer Dilated causal convolutions [7] with $d \in \{1, 2\}$ extract local patterns. The TCN branch uses causal convolutions; the subsequent Transformer encoder remains bidirectional, as intended by the masked-PLL objective:

$$\mathbf{y}(t) = (\mathbf{x} *_d f)(t) = \sum_{i=0}^{k-1} f(i) \cdot \mathbf{x}(t - d \cdot i) \quad (2)$$

$$\mathbf{H}_{\text{TCN}} = \text{LayerNorm}(\text{Proj}(\mathbf{E}) + \text{GELU}(\text{CausalTCN}(\mathbf{E}))) \quad (3)$$

Transformer Encoder and Scoring A single Transformer encoder layer (pre-LN, multi-head attention, GELU FFN) contextualizes the sequence. Training minimizes masked cross-entropy. Let $\mathcal{V}$ be the set of non-padding positions and

$$\mathbf{X}^{(-t)} = [T_1, \dots, [\text{MASK}]_t, \dots, T_L]. \quad (4)$$

Inference uses leave-one-position-out PLL:

$$S(\mathbf{X}) = \frac{1}{|\mathcal{V}|} \sum_{t \in \mathcal{V}} -\log P(T_t | \mathbf{X}^{(-t)}). \quad (5)$$

The primary threshold is $\tau = \mu_{\text{val}} + 3\sigma_{\text{val}}$ on nominal validation scores; as a diagnostic we also report a validation-selected F1 threshold applied to test scores.

4 Experimental Evaluation

4.1 Corpus and Window Labels

We stream selective June daily backup archives from a local institutional campus extract, focusing on application/service syslog members from aca, careerhub, and courses. We skip `fortigate.csv` (heterogeneous firewall CSV schema; not comparable to the syslog template stream without a separate parser) and members >500 MiB (computational bound for the selective ingest). Protocol B keeps the first 1M unique raw lines under per-day and per-file caps (200k lines/day; 40k/file). Proxy line labels use HTTP 4xx/5xx, deny/block fields, and keywords—not SOC adjudication. A window is positive iff *any* of its L lines is positive (OR aggregation). Under independence, $1 - (1 - p)^L$ with $p=0.0796$ predicts prevalence ≈ 0.81 , matching Protocol A’s 80.5% test rate.

4.2 Protocol A (archive-order split with validation contamination)

Records are ordered by backup day $\rightarrow$ ZIP $\rightarrow$ member $\rightarrow$ in-member position, then split 80/10/10 before windowing. This is *not* event time: windows may cross file/service boundaries. A raw-line hash audit found 36.235% of unique validation lines also in training and 11.382% of unique test lines also occur in validation; train/test exact raw-line overlap is only 0.0176% of unique test lines (13 shared hashes), so Protocol A’s primary risk is archive-order structure and *validation contamination* (validation is used for threshold calibration) rather than massive train $\rightarrow$ test line leakage. Exact template-window overlap is likewise small on test (13 of 7,573 unique test windows). Results: `huflit.baselines.json`.

4.3 Protocol B (dedup + source-bounded + backup-day holdout)

Protocol B *jointly* applies exact-line deduplication, boundary-safe windowing, and backup-day holdout; we therefore treat A $\rightarrow$ B as a comparison of protocols, not a causal ablation of archive ordering alone. We keep the first SHA-1 of each *raw line* (262,194 duplicates skipped while filling 1M unique lines), emit windows only inside one (backup-day, source, member) group, and assign backup days 15–17 Jun $\rightarrow$ train, 18 Jun $\rightarrow$ val, 19 Jun $\rightarrow$ test. Day folders are backup collection periods, not verified per-event timestamps. Test-window prevalence is 56.36% (always-anomaly F1 baseline 0.721).

Global exact-line deduplication may also remove legitimate repeated events that recur identically across backup days; we therefore treat it as a *conservative decontamination* step that can alter the backup-day-held-out test distribution, and report this as an additional validity limitation rather than a claim that every duplicate is a backup artifact.

Raw-line dedup does *not* imply unique template windows. Exact template-ID window sequence overlap (SHA-1 of the integer window vector): 505 of 11,065 distinct validation windows (4.56%) also occur in training, and 110 of 11,137 distinct test windows (0.99%) also occur in training (`huflit_v2_split_meta.json`).

4.4 Experimental Setup

Table 2 lists corpus settings for Protocol B. Table 3 summarizes shared baseline configurations. Protocol A uses the same architectures on the earlier stream-position arrays (`data/processed/huflit`). Training is nominal-only (proxy labels select the nominal subset and the nominal validation scores used for $\mu+3\sigma$); seeds {42, 101, 202, 303, 404}; hardware is CPU (PyTorch 2.8.0+cpu, 2 intra-op threads).

Table 2. Protocol B corpus setup. Window label = OR over L proxy line labels.

Item	Value
Unique lines kept / dups skipped	1,000,000 / 262,194
Train / val / test windows	59,966 / 19,988 / 19,990
Test positives / prevalence	11,267 / 56.36%
Always-anomaly F1 baseline (test)	0.721
Vocab (templates+pad+mask)	1,183
L , stride	20, 10
Seeds	42, 101, 202, 303, 404
Hardware	CPU, 2 PyTorch threads

Table 3. Baseline and masked-model configuration (both protocols unless noted).

Component	Configuration
PCA	Count vectors; $n=32$ components (capped); sklearn SVD; score = reconstruction MSE.
DeepLog-style*	Emb 64, LSTM hidden 128, 2 layers; next-token CE; score = mean next-token NLL; Adam 10^{-3} ; batch 256; 6 epochs.
LogAnomaly-lite	Emb+2-layer LSTM recon on template IDs only (no template2Vec; no quantitative branch); score = mean token NLL; same optimizer.
Masked Transformer	$d=128$, heads 4, 1 layer, FFN 256, mask 15%; score = leave-one-out PLL.
Masked TCN-Transformer	As above + causal conv $k=3$, dilations {1, 2}; same PLL score.
Primary / diagnostic thr.	$\mu_{\text{val}}+3\sigma_{\text{val}}$ on nominal val; val-F1 oracle applied to <i>test</i> scores (diagnostic).

*Our DeepLog-style reimplementation uses mean next-token NLL for continuous ranking and threshold evaluation rather than the original top- g decision rule [2].

4.5 Models and Metrics

We evaluate PCA count-vector reconstruction [6], a DeepLog-style LSTM [2], LogAnomaly-lite [3], a masked Transformer without TCN, and the masked TCN-

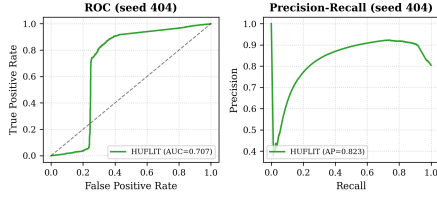


Fig. 2. Protocol A ROC/PR for seed 404 (illustrative; one of five evaluation seeds; tables report mean $\pm$ std).

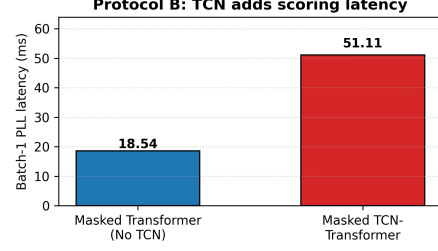


Fig. 3. Protocol B batch-1 PLL latency: Masked Transformer (No TCN) vs. Masked TCN-Transformer.

Transformer under *both* protocols on the same window arrays within each protocol. Metrics: F1/precision/recall at $\mu+3\sigma$, ROC-AUC, AUPRC, **AUPRC lift over prevalence** (interpreted *within* each protocol, not as a cross-prevalence normalized score), always-anomaly F1 baseline, and batch-1 PLL latency (masked models only).

Table 4. Protocol A (archive-order stream-position with validation contamination): $\tau = \mu+3\sigma$; test prevalence 80.5%; always-anomaly F1 baseline $\approx$ 0.892; AUPRC no-skill $\approx$ 0.805

Model	F1	ROC-AUC	AUPRC	Lift	Batch-1 ms
Always-anomaly (trivial)	0.892	0.500	0.805	0.000	—
PCA	0.000 $\pm$ 0.000	0.738 $\pm$ 0.004	0.913 $\pm$ 0.003	+0.108	NR
DeepLog-style	0.011 $\pm$ 0.022	0.781 $\pm$ 0.020	0.882 $\pm$ 0.025	+0.077	NR
LogAnomaly-lite	0.010 $\pm$ 0.013	0.777 $\pm$ 0.045	0.882 $\pm$ 0.040	+0.077	NR
Masked Transformer (No TCN)	0.348 $\pm$ 0.410	0.769 $\pm$ 0.035	0.870 $\pm$ 0.036	+0.065	2.04
Masked TCN-Transformer	0.040 $\pm$ 0.016	0.723 $\pm$ 0.018	0.836 $\pm$ 0.014	+0.031	52.92

Table 5. Protocol B (dedup + source-bounded + backup-day holdout): primary $\tau = \mu+3\sigma$; test prevalence 56.4%; always-anomaly F1 baseline 0.721

Model	F1@3 σ	ROC-AUC	AUPRC	Lift	Batch-1 ms
Always-anomaly (trivial)	0.721	0.500	0.564	0.000	—
PCA	0.000 $\pm$ 0.000	0.629 $\pm$ 0.008	0.661 $\pm$ 0.006	+0.098	—
DeepLog-style	0.013 $\pm$ 0.027	0.657 $\pm$ 0.134	0.702 $\pm$ 0.153	+0.138	NR
LogAnomaly-lite	0.385 $\pm$ 0.293	0.632 $\pm$ 0.109	0.676 $\pm$ 0.131	+0.112	NR
Masked Transformer (No TCN)	0.001 $\pm$ 0.002	0.493 $\pm$ 0.009	0.511 $\pm$ 0.007	-0.052	18.54 $\pm$ 0.47
Masked TCN-Transformer	0.151 $\pm$ 0.183	0.532 $\pm$ 0.112	0.557 $\pm$ 0.124	-0.007	51.11 $\pm$ 9.08

4.6 Results

On Protocol A, the DeepLog-style LSTM leads ROC-AUC (0.781); LogAnomaly-lite is close (0.777). The TCN is *worse* than no-TCN on mean F1/AUC and much slower at batch-1 PLL. The always-anomaly F1 baseline (0.892) exceeds every learned F1 at $\mu+3\sigma$; AUPRC must be read against prevalence 0.805.

On Protocol B (Table 5), DeepLog-style and LogAnomaly-lite retain the strongest mean AUPRC lifts (+0.138 and +0.112), but with large seed variance (DeepLog-style ROC-AUC 0.657 $\pm$ 0.134). Masked models lose backup-day ranking (AUC $\approx$ 0.49–0.53). PCA keeps a smaller but more stable lift (+0.098).

Table 6. Protocol B keyword/status template ablation: 167 trigger templates ($\approx 9.3\%$ of test tokens) are remapped to padding before scoring

Model	ROC-AUC	AUPRC lift	Δ lift vs. full
PCA (full)	0.629 ± 0.008	+0.098	—
PCA (ablated)	0.626 ± 0.007	+0.084	−0.014
Masked Transformer (full)	0.493 ± 0.009	−0.052	—
Masked Transformer (ablated)	0.494 ± 0.005	−0.050	+0.002

Table 7. Protocol B threshold comparison (mean over 5 seeds): fixed $\mu+3\sigma$ vs. *test* metrics at a validation-selected F1 threshold τ_{val} (diagnostic)

Model	F1@ 3σ	P@ 3σ	R@ 3σ	F1@ τ_{val}	P@ τ_{val}	R@ τ_{val}
PCA	0.000	0.000	0.000	0.729	0.585	0.966
DeepLog-style	0.013	0.423	0.007	0.745	0.619	0.934
LogAnomaly-lite	0.385	0.625	0.297	0.742	0.601	0.969
Masked Transformer	0.001	0.152	0.000	0.745	0.620	0.934
Masked TCN-Transformer	0.151	0.354	0.102	0.741	0.608	0.949

Table 7: $\mu+3\sigma$ starves recall for most detectors (PCA/DeepLog-style/masked Transformer recall ≈ 0 –0.01), while thresholds selected on validation and evaluated on *test* recover F1 ≈ 0.73 –0.75—near the always-anomaly test baseline (0.721)—showing that much of the apparent F1 gain is driven by prevalence and threshold calibration rather than strong ranking quality. Under Protocol B, TCN remains slower (51 vs. 19 ms) without consistent accuracy gains.

Table 6: remapping 167 proxy-trigger templates ($\approx 9.3\%$ of test tokens) to padding reduces PCA lift from +0.098 to +0.084 without collapsing ranking to chance, while the masked Transformer stays near chance before and after ablation. Trigger templates are those whose template string matches a status/keyword pattern used by the proxy-labeling rules (HTTP 4xx/5xx and deny/block/error-like tokens). For the masked models, remapping to padding removes these positions from attention and from the PLL average rather than introducing a dedicated ablation token; for PCA, pad IDs are simply omitted from the count vector. Residual correlation remains after template-level trigger ablation; without SOC labels, it cannot be interpreted as validated operational anomaly signal.

Due to privacy restrictions, raw campus telemetry cannot be redistributed. We retain code, configuration, aggregate split statistics, and derived evaluation JSON artifacts in the project repository for co-author reproduction; a public mirror is contingent on institutional approval.

5 Discussion & Threats to Validity

What sequence means: Protocol A orders archive members, not wall-clock events. Protocol B jointly removes cross-member windows and duplicate raw lines but still uses backup-day folders rather than verified timestamps. Seed

variance does not capture day-to-day sampling uncertainty because Protocol B contains a single held-out backup day.

Window OR labels: OR aggregation manufactures high prevalence and makes the always-anomaly F1 baseline a necessary reference (not an upper bound: a perfect detector still has $F1=1$). Ranking metrics and lift-over-prevalence must accompany thresholded F1; raw AUPRC lift is interpreted within each protocol and not directly compared across prevalences.

Proxy labels and label–feature coupling: HTTP status tokens, deny/block fields, and keyword heuristics both *define* proxy labels and often remain visible in template text. Proxy labels also select nominal-only training windows and the nominal validation scores used for $\mu+3\sigma$. Models may therefore learn the labeling rule rather than operational anomalies. Our keyword/status template ablation (Table 6) supports partial leakage without claiming residual operational signal. Gold SOC labels remain required before operational claims.

Parser: The custom parser has no gold template annotations; the 0.5 merge threshold is untested for sensitivity, and parser merge errors may alter event frequency and sequence structure [9].

Corpus selection: Skipping FortiGate CSV and >500 MiB members is a deliberate selective-ingest policy (schema mismatch and compute bound), not a claim of full-archive coverage.

Ethics: Campus telemetry stays private; any future release needs redaction and authorization.

6 Conclusion

We showed that OR window labels can inflate prevalence and make thresholded F1 look stronger than ranking warrants on HUFLIT campus logs, and that a jointly decontaminated backup-day-held-out protocol changes conclusions relative to an archive-ordered stream split. Masked TCN/Transformer ranking collapses under Protocol B while PCA and classical sequence baselines retain unstable or modest AUPRC lifts; TCN adds latency without a reliable benefit. Keyword/status template ablation only partially reduces PCA lift. Future work: factorized protocol ablations, additional backup-day folds, event timestamps, SOC labels, and Loghub [11] sanity checks under the same harness.

Acknowledgment

The authors thank the Faculty of Information Technology, Ho Chi Minh City University of Foreign Languages - Information Technology (HUFLIT), Ho Chi Minh City, Vietnam for providing the production campus log telemetry and computing infrastructure. Generative AI tools were used for literature discovery, consistency checking, and language refinement. All cited sources were independently verified by the authors. Study design, implementation, experimental execution, statistical analysis, interpretation, and final scholarly judgments were performed and approved by the authors.

Disclosure of Interests

The authors have no competing interests to declare that are relevant to the content of this article.

References

1. He, P., Zhu, J., Zheng, Z., Lyu, M.R.: Drain: An Online Log Parsing Approach with Fixed Depth Tree. In: Proc. IEEE Int. Conf. Web Serv. (ICWS), pp. 33–40 (2017).
2. Du, M., Li, F., Zheng, G., Srikumar, V.: DeepLog: Anomaly Detection and Diagnosis from System Logs through Deep Learning. In: Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS), pp. 1285–1298 (2017).
3. Meng, W., Liu, Y., Zhu, Y., et al.: LogAnomaly: Unsupervised Detection of Sequential and Quantitative Anomalies in Unstructured Logs. In: Proc. 28th Int. Joint Conf. Artif. Intell. (IJCAI), pp. 4739–4745 (2019). <https://doi.org/10.24963/ijcai.2019/658>
4. Zhang, X., Xu, Y., Lin, Q., et al.: Robust Log-Based Anomaly Detection on Unstable Log Data. In: Proc. 27th ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE), pp. 807–817 (2019).
5. Li, X., Chen, P., Jing, L., He, Z., Yu, G.: SwissLog: Robust Anomaly Detection and Localization for Interleaved Unstructured Logs. IEEE Trans. Dependable Secure Comput. 20(4), 2762–2780 (2023). <https://doi.org/10.1109/TDSC.2022.3162887>
6. Xu, W., Huang, L., Fox, A., Patterson, D., Jordan, M.I.: Mining Console Logs for Large-Scale System Problem Detection. In: Proc. 3rd Workshop on Tackling Computer Systems Problems with Machine Learning Techniques (SysML), (2008).
7. Bai, S., Kolter, J.Z., Koltun, V.: An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. arXiv preprint arXiv:1803.01271 (2018).
8. Vaswani, A., Shazeer, N., Parmar, N., et al.: Attention Is All You Need. In: Adv. Neural Inf. Process. Syst. (NeurIPS), vol. 30, pp. 5998–6008 (2017).
9. Le, V.H., Zhang, H.: Log-based Anomaly Detection with Deep Learning: How Far Are We? In: Proc. 44th Int. Conf. Softw. Eng. (ICSE), pp. 1356–1367 (2022). <https://doi.org/10.1145/3510003.3510155>
10. Albert, R.G.: System Logs Anomaly Detection. Are we on the right path? Appl. Artif. Intell. 39(1) (2025). <https://doi.org/10.1080/08839514.2024.2440692>
11. Zhu, J., He, S., He, P., Liu, J., Lyu, M.R.: Loghub: A Large Collection of System Log Datasets for AI-driven Log Analytics. In: Proc. 34th IEEE Int. Symp. Softw. Reliab. Eng. (ISSRE), pp. 355–366 (2023).
12. Du, M., Li, F.: Spell: Streaming Parsing of System Event Logs. In: Proc. IEEE Int. Conf. Data Min. (ICDM), pp. 859–864 (2016).
13. Liu, M., Wang, T., Zhao, Y., Xia, C., Zeng, Y., Tao, Y.: LogMUSE: Log Anomaly Detection via Multi-Scale Semantic Representation. IEEE Trans. Serv. Comput. 19(4), 2903–2916 (2026). <https://doi.org/10.1109/TSC.2026.3700403>
14. Dietz, S., Klede, K., Nguyen, A., Eskofier, B.M.: ContraLog: Log File Anomaly Detection with Contrastive Learning and Masked Language Modeling. arXiv preprint arXiv:2602.03678 (2026).
15. Liao, N., Liu, Z.: Log Anomaly Detection Method Based on Transformer and Temporal Convolutional Networks. IEEE Access 13, 68547–68560 (2025). <https://doi.org/10.1109/ACCESS.2025.3561669>